RTOS Implementation — Battery Monitoring System

By: Dhanush Raja A K

Summary: Interrupt-Based vs RTOS-Based

The current design already uses interrupts (CAN RX, and typically a timer interrupt for periodic sampling), and that doesn't go away under RTOS, it's actually still the entry point for time-critical events. What changes is what happens *after* the interrupt fires.

Aspect	Pure Interrupt-Based	RTOS-Based
Where the work happens	Directly inside the ISR	ISR just signals a task; the task does the real work
ISR duration	Can grow long as logic is added (fault checks, CAN packing, etc.)	Stays minimal and constant, regardless of how much logic exists
Priority between jobs	Whichever interrupt fires first wins; no concept of "importance"	Explicit priorities, a fault check can always preempt an LCD update
Adding a new feature	Risk of extending ISR time, which can delay or block other interrupts	New task with its own priority, doesn't affect existing ISR timing
Shared data between jobs	Manual disable/enable interrupts around shared variables	Queues and mutexes handle this safely, no manual interrupt masking
Timeout / watchdog logic	Usually bolted on inside the main loop or an interrupt	Runs as its own task with a guaranteed period
Debugging timing issues	Hard to see what ran when, ISR nesting can be tricky to trace	RTOS-aware debuggers show task states, timing, and queue depth directly
Code as project grows	Tends to become one large ISR or a tangled main loop	Stays modular, each job is still a separate, independently testable task

The short version: interrupts still capture the event the instant it happens, same as now. RTOS just moves the *processing* of that event out of interrupt context and into a scheduled task, so you get priority control, safer data sharing, and code that stays manageable as more fault checks and features get added. Nothing about your existing interrupt setup needs to be thrown away, it gets wrapped rather than replaced.

Why RTOS

Both STM1 and STM2 currently handle their time-sensitive work through interrupts, CAN RX for incoming data, and typically a timer interrupt for periodic sensor sampling. That works, but it starts to strain once several time-sensitive jobs need to coexist: sampling every 10 ms, validating and transmitting every 100 ms, watching for a CAN timeout, and updating an LCD, all without one of them blocking or delaying another. As more logic gets added directly into ISRs, they grow longer and start risking missed or delayed interrupts elsewhere.

An RTOS (this document assumes **FreeRTOS**, since it's what ships with STM32Cube) addresses that by keeping interrupts as the trigger, but moving the actual processing into independent **tasks**, each with its own priority and timing, coordinated by a **scheduler**. The interrupt still fires exactly when it did before; it just hands off to a task instead of doing everything itself. A high-priority fault check will always preempt a lower-priority LCD update, which is exactly the behavior we want for a safety-relevant system.

Design Principles

- Each task does one job. Sensor reading, validation, CAN handling, fault checking, and output control are all separate tasks.
- Tasks never share data directly. They communicate through **queues** or protected variables

(**mutexes**), never by touching each other's variables directly.

- Priority reflects urgency, not importance. Fault detection and CAN handling run higher than LCD updates, since a slow LCD refresh is harmless but a missed fault check is not.
- Nothing blocks forever. Every wait has a timeout, so one stuck task can't silently freeze the whole system.

STM1 — Task Architecture (Sensor Node)

STM1's job is narrow: read sensors, validate, filter, and transmit. That maps to three tasks.

Task	Priority	Period	Purpose
`Task_ReadSensors`	Medium	10 ms	Reads ADC/sensor values, keeps a rolling average
`Task_Validate`	Medium	100 ms	Range-checks values, sets fault flags
`Task_CANSend`	High	100 ms	Packs and transmits the CAN frame

Data Flow

```
Task_ReadSensors  --(queue)-->  Task_Validate  --(queue)-->  Task_CANSend
```

Each arrow is a FreeRTOS queue, not a shared global. `Task_ReadSensors` pushes raw samples in, `Task_Validate` pulls them out, checks the ranges from the earlier validation logic, and pushes a clean struct forward.

Shared Data Structure

```
typedef struct {
    float temperature;
    float voltage;
    float current;
    uint16_t smoke;
    uint8_t fault_flags;   // bit0=temp, bit1=voltage, bit2=current, bit3=smoke
} SensorData_t;

QueueHandle_t xSensorQueue;   // raw samples: Task_ReadSensors -> Task_Validate
QueueHandle_t xValidatedQueue; // validated data: Task_Validate -> Task_CANSend
```

Task_ReadSensors

Samples every 10 ms and keeps a running average of the last 10 readings per sensor, this is the noise-filtering step from the STM1 sensor logic, just moved into its own task so it doesn't block anything else.

```
void Task_ReadSensors(void *pvParameters)
{
    SensorData_t raw;
    TickType_t xLastWakeTime = xTaskGetTickCount();

    for (;;)
    {
        raw.temperature = Read_Temperature();
        raw.voltage     = Read_Voltage();
        raw.current     = Read_Current();
        raw.smoke       = Read_ADC();

        xQueueSend(xSensorQueue, &raw, pdMS_TO_TICKS(5));

        vTaskDelayUntil(&xLastWakeTime, pdMS_TO_TICKS(10));
    }
}
```

`vTaskDelayUntil` is used instead of `vTaskDelay` so the sampling period stays fixed at 10 ms regardless of how long the task itself takes to run.

Task_Validate

Wakes every 100 ms, pulls the latest averaged sample, and runs the same range checks from the sensor validation logic.

```
void Task_Validate(void *pvParameters)
{
    SensorData_t raw, validated;

    for (;;)
    {
        if (xQueueReceive(xSensorQueue, &raw, pdMS_TO_TICKS(100)) == pdPASS)
        {
            validated = raw;
            validated.fault_flags = 0;

            if (raw.temperature < -20 || raw.temperature > 100)
                validated.fault_flags |= (1 << 0);

            if (raw.voltage < 0 || raw.voltage > 20)
                validated.fault_flags |= (1 << 1);

            if (raw.current < -15 || raw.current > 15)
                validated.fault_flags |= (1 << 2);

            if (raw.smoke > 4095)
                validated.fault_flags |= (1 << 3);

            xQueueSend(xValidatedQueue, &validated, pdMS_TO_TICKS(10));
        }
    }
}
```

Task_CANSend

Highest priority on STM1, since a delayed CAN message is what triggers STM2's timeout fault. Applies the change-detection logic so it isn't flooding the bus with values that haven't moved.

```
void Task_CANSend(void *pvParameters)
{
    SensorData_t data, last_sent = {0};

    for (;;)
    {
        if (xQueueReceive(xValidatedQueue, &data, pdMS_TO_TICKS(100)) == pdPASS)
        {
            if (fabs(data.voltage - last_sent.voltage) > 0.05 ||
                data.fault_flags != last_sent.fault_flags)
            {
                Pack_CAN_Frame(&data);
                HAL_CAN_AddTxMessage(&hcan1, &TxHeader, TxData, &TxMailbox);
                last_sent = data;
            }
        }
    }
}
```

STM2 — Task Architecture (Fault Detection & Output)

STM2 receives data, calculates derived values, checks faults, and drives outputs. That's naturally four tasks plus a CAN receive interrupt.

Task	Priority	Trigger	Purpose
`Task_CANReceive`	Highest	CAN interrupt	Pulls incoming frames off the bus fast
`Task_FaultDetect`	High	100 ms	Runs all fault checks, sets priority fault
`Task_OutputControl`	High	On fault change	Drives fan, buzzer, GPIO
`Task_LCDUpdate`	Low	500 ms	Refreshes the display
`Task_Watchdog`	High	100 ms	Tracks CAN timeout, feeds hardware watchdog

Data Flow

```
CAN RX Interrupt --> Task_CANReceive --(queue)--> Task_FaultDetect --(queue)--> Task_Outpu
                                                                    |
                                                                    +--(mutex)--> Task_LCDUpdate
```

Task_CANReceive

Runs at the highest priority so incoming frames are pulled off the hardware buffer immediately, this prevents overruns if STM1 sends faster than STM2 processes.

```
void Task_CANReceive(void *pvParameters)
{
    SensorData_t incoming;

    for (;;)
    {
        if (xSemaphoreTake(xCANRxSemaphore, portMAX_DELAY) == pdTRUE)
        {
            Unpack_CAN_Frame(RxData, &incoming);
            xQueueOverwrite(xLatestDataQueue, &incoming);
            xLastCANReceiveTime = xTaskGetTickCount();
        }
    }
}
```

`xCANRxSemaphore` is given from the `HAL_CAN_RxFifo0MsgPendingCallback` ISR, the same interrupt callback used today. The only change is what's inside it: instead of unpacking and processing the frame right there in interrupt context, it now just calls `xSemaphoreGiveFromISR()` and returns. All the actual work moves into `Task_CANReceive`, running at a controlled priority instead of interrupt priority.

`xQueueOverwrite` is used in the task (not a regular send) because only the *latest* value matters here, there's no benefit to queuing up stale sensor readings.

Task_FaultDetect

The core logic from the fault detection layer, now running as its own periodic task so it always executes on schedule regardless of what the LCD or CAN tasks are doing.

```
void Task_FaultDetect(void *pvParameters)
{
    SensorData_t data;
    FaultState_t fault;
    TickType_t xLastWakeTime = xTaskGetTickCount();

    for (;;)
    {
        xQueuePeek(xLatestDataQueue, &data, 0);

        fault.under_voltage  = (data.voltage < 12.5);
        fault.over_voltage   = (data.voltage > 16.8);
        fault.over_current   = (data.current > 8.0);
        fault.over_temp      = (data.temperature >= 45);
```

```
        fault.critical_temp  = (data.temperature >= 55);
        fault.smoke          = (data.smoke > 250);
        fault.can_timeout    = ((xTaskGetTickCount() - xLastCANReceiveTime) > pdMS_TO_TICK
        fault.can_busoff     = (HAL_CAN_GetError(&hcan1) & HAL_CAN_ERROR_BOF);
        fault.sensor_fail    = (data.fault_flags != 0);

        fault.active = Select_Highest_Priority(&fault);

        xQueueOverwrite(xFaultQueue, &fault);

        vTaskDelayUntil(&xLastWakeTime, pdMS_TO_TICKS(100));
    }
}
```

`Select_Highest_Priority()` walks the priority table from the fault detection document (smoke first, sensor failure last) and returns a single active fault code for the other tasks to act on.

Task_OutputControl

Reacts as soon as a new fault state is posted, rather than polling on a fixed timer, since fan and buzzer response time matters for safety.

```
void Task_OutputControl(void *pvParameters)
{
    FaultState_t fault;

    for (;;)
    {
        if (xQueueReceive(xFaultQueue, &fault, portMAX_DELAY) == pdTRUE)
        {
            Apply_Fan_State(fault.active);
            Apply_Buzzer_Pattern(fault.active);
            xSemaphoreGive(xLCDDataMutex);
        }
    }
}
```

Task_LCDUpdate

Deliberately low priority and slow (500 ms), since redrawing a screen is the least time-critical job in the system and shouldn't be allowed to delay a fault response.

```
void Task_LCDUpdate(void *pvParameters)
{
    FaultState_t fault;
    SensorData_t data;

    for (;;)
    {
        if (xSemaphoreTake(xLCDDataMutex, pdMS_TO_TICKS(500)) == pdTRUE)
        {
            xQueuePeek(xFaultQueue, &fault, 0);
            xQueuePeek(xLatestDataQueue, &data, 0);
            LCD_Render(&fault, &data);
        }
        vTaskDelay(pdMS_TO_TICKS(500));
    }
}
```

Task_Watchdog

A separate lightweight task that feeds the hardware independent watchdog (IWDG), but only if the other critical tasks are still alive. Each critical task checks in by setting a bit; if any bit is missing, the watchdog is deliberately **not** fed, forcing a hardware reset instead of letting a hung task run silently.

```
void Task_Watchdog(void *pvParameters)
{
    for (;;)
    {
        if ((xTaskAliveFlags & REQUIRED_TASK_MASK) == REQUIRED_TASK_MASK)
        {
            HAL_IWDG_Refresh(&hiwdg);
            xTaskAliveFlags = 0;
        }
        vTaskDelay(pdMS_TO_TICKS(100));
    }
}
```

Priority Summary

FreeRTOS priorities are relative, higher number runs first when tasks are ready at the same time. The table below lines up both boards on one scale for reference.

Board	Task	Priority	Reasoning
STM2	`Task_CANReceive`	5 (highest)	Must never miss or delay an incoming frame
STM2	`Task_Watchdog`	4	Safety net, must run even under heavy load
STM1	`Task_CANSend`	4	Directly affects STM2's timeout detection
STM2	`Task_FaultDetect`	4	Time-critical, drives fan/buzzer decisions
STM2	`Task_OutputControl`	3	Reacts to fault changes, not itself time-critical
STM1	`Task_ReadSensors`	3	Needs a steady 10 ms cadence but tolerates jitter
STM1	`Task_Validate`	2	Runs after sampling, not urgent
STM2	`Task_LCDUpdate`	1 (lowest)	Purely cosmetic, can lag without consequence

---

Synchronization Objects Used

Object	Type	Used For
`xSensorQueue`	Queue	Raw samples from `Task_ReadSensors` to `Task_Validate`
`xValidatedQueue`	Queue	Validated data from `Task_Validate` to `Task_CANSend`
`xLatestDataQueue`	Queue (1)	Most recent CAN data, overwritten each receive
`xFaultQueue`	Queue (1)	Current fault state, overwritten each check
`xCANRxSemaphore`	Semaphore	Signals `Task_CANReceive` from the CAN RX interrupt
`xLCDDataMutex`	Mutex	Protects LCD read access from a torn read mid-update

A queue of length 1 combined with `xQueueOverwrite` is used wherever only the *latest* value matters, this avoids a backlog of stale data building up if a consumer task briefly falls behind.

Stack Sizing Guidance

These are starting points, actual usage should be checked with `uxTaskGetStackHighWaterMark()` during testing and adjusted from there.

Task	Suggested Stack (words)
`Task_CANReceive`	256
`Task_FaultDetect`	256

`Task_OutputControl`	128
`Task_LCDUpdate`	256
`Task_Watchdog`	128
`Task_ReadSensors`	192
`Task_Validate`	192
`Task_CANSend`	192

Why This Structure Holds Up

The single-loop version of this system works until two things need attention at the same moment, at which point one of them waits, even if it’s the fault check. Splitting the work into tasks with the priorities above means:

- A fault is always detected and acted on within one 100 ms cycle, regardless of what the LCD is doing.
- A slow or blocked LCD update can never delay the fan or buzzer response.
- STM1’s CAN transmission stays on schedule even while sensor sampling and validation are running underneath it.
- If any critical task hangs, the watchdog task stops feeding the IWDG and the board resets instead of failing silently.

This is the same reasoning automotive and industrial control systems use RTOS for: not raw performance, but **predictable timing** on the tasks that matter most.